Basic Information

Experiment No.: 12

Date: 11-10-2026

Title: Multithreading – Synchronized Seat Reservation System

Objective / Aim

To implement a seat reservation system using multiple threads and synchronization, ensuring that only one thread accesses the "reserve()" method at a time.

System Requirements

Hardware

- Computer/Laptop
- Minimum 4 GB RAM

Software

- Java JDK
- IntelliJ IDEA or any Java IDE
- Operating System: Windows/Linux

Algorithm / Logic Description

1. Create a reservation class with 100 available seats.
2. Define a synchronized "reserve()" method.
3. Accept the person name and requested seats.
4. Check whether enough seats are available.

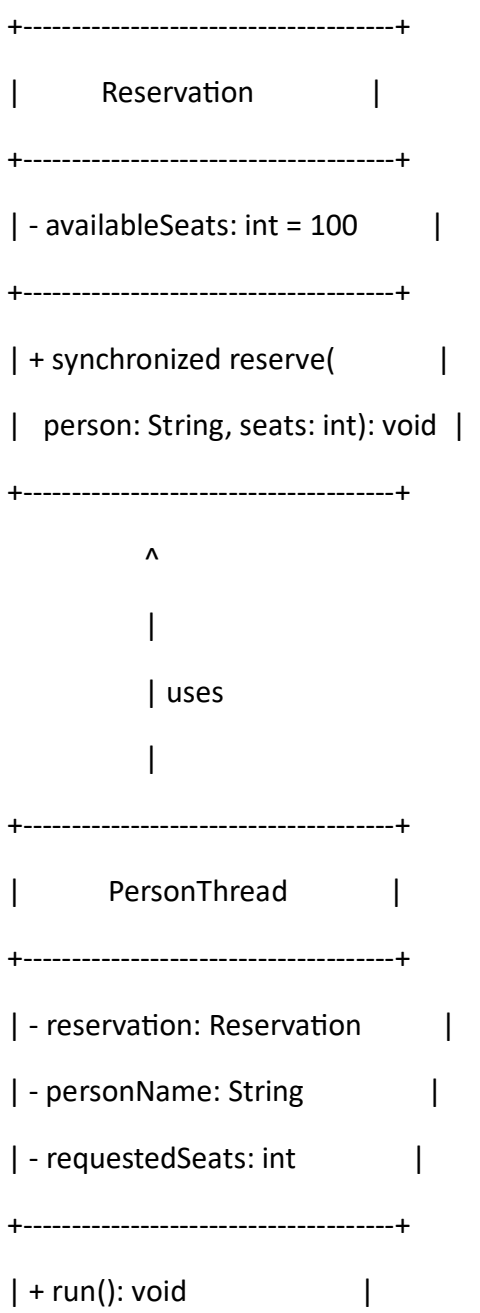
5. If available, reserve the seats and update the count.

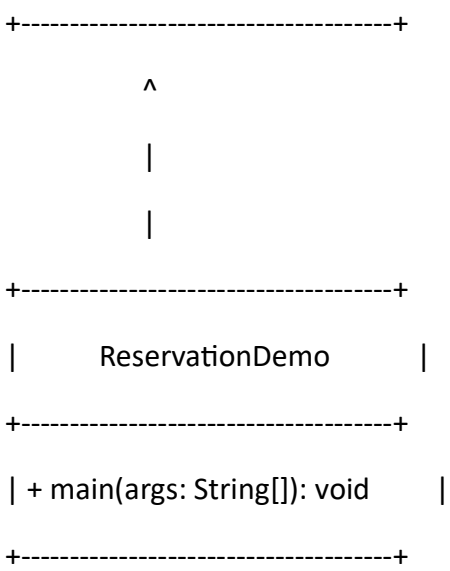
6. If unavailable, display a message.

7. Create multiple person threads to book seats.

8. Start the threads and display the reservation results.

📐 UML Class Diagram





💻 Source Code

```
class Reservation {

   // Initially, 100 seats are available
   private int availableSeats = 100;

   // Only one thread can execute this method at a time
   public synchronized void reserve(String person, int seats) {

      System.out.println(person + " entered.");
      System.out.println("Available seats: " + availableSeats
            + " Requested seats: " + seats);

      // Check whether the requested seats are available
      if (seats <= availableSeats) {
         System.out.println("Seat Available. Reserve now :-)");
```

```
            availableSeats -= seats;

            System.out.println(seats + " seats reserved.");
        } else {
            System.out.println("Requested seats not available :-)");
        }

        System.out.println(person + " leaving.");
        System.out.println("-------------------------------------");
    }
}

class PersonThread extends Thread {

    private Reservation reservation;
    private int requestedSeats;

    // Initialize the reservation object and requested seats
    public PersonThread(Reservation reservation,
                        String name, int requestedSeats) {
        super(name);
        this.reservation = reservation;
        this.requestedSeats = requestedSeats;
    }

    // Thread execution starts here
    public void run() {
```

```
        reservation.reserve(getName(), requestedSeats);
    }
}

public class ReservationDemo {

    public static void main(String[] args) {

        // Create one shared reservation system
        Reservation reservation = new Reservation();

        // Create person threads
        PersonThread p1 =
                new PersonThread(reservation, "Person-1", 5);

        PersonThread p2 =
                new PersonThread(reservation, "Person-2", 2);

        PersonThread p3 =
                new PersonThread(reservation, "Person-3", 4);

        // Start all threads
        p1.start();
        p2.start();
        p3.start();
    }
}
```

🔨 Compilation & Execution Commands

Save the file as:

ReservationDemo.java

Compile:

javac ReservationDemo.java

Run:

java ReservationDemo

📟 Output Screen

Person-1 entered.

Available seats: 100 Requested seats: 5

Seat Available. Reserve now :-)

5 seats reserved.

Person-1 leaving.

Person-2 entered.

Available seats: 95 Requested seats: 2

Seat Available. Reserve now :-)

2 seats reserved.

Person-2 leaving.

Person-3 entered.

Available seats: 93 Requested seats: 4

Seat Available. Reserve now :-)

4 seats reserved.

Person-3 leaving.

Note: The order of thread execution can vary. With 100 initial seats and requests of 5, 2, and 4, all three reservations succeed. The sample image's values of 10, 5, 2, and 4 are illustrative; this program follows the question's requirement of initially having 100 seats.

📌 Result

The multithreaded seat reservation system was successfully implemented using Java threads and a synchronized "reserve()" method. Synchronization ensures that only one thread can reserve seats at a time.